Proper forest-based recognition is NP-complete for binary two-root networks

Working manuscript — awaiting expert review
16 September 2026

Abstract
A proper base forest of a rooted network has as many tree components as the network has roots. Recognition is polynomial for binary tree-child networks with two roots, whereas the complexity for general binary two-root networks was left open. We give a deterministic Karp reduction from 3-Uniform Set Splitting. Binary reticulation stacks realize a not-all-equal constraint, and private leaves let binary trees copy colors without constraining their recipients. The construction has linear graph size and admits polynomial-time recovery of a source split from every valid target forest. Consequently, proper forest-based recognition is NP-complete for general binary networks with exactly two roots.

1. Introduction
Proper forest-based networks represent several rooted trees within one directed network. The word "proper" fixes the number of forest components to the number of network roots. This constraint makes the number of roots a natural boundary in recognition complexity.

Huber et al. [1, Theorem 4.1] prove NP-completeness for quasi-binary tree-child networks with two roots. Their construction allows indegree-three reticulations. For binary tree-child networks, recognition is NP-complete with at least three roots [1, Theorem 6.1], but polynomial with two roots [1, Theorem 6.2]. Section 8 of that paper leaves recognition for general binary two-root networks open.

Theorem 1. Proper forest-based recognition is NP-complete on weakly connected binary networks with exactly two roots. There is a deterministic polynomial-time many-one reduction from 3-Uniform Set Splitting, with linear graph size, together with a polynomial-time algorithm that recovers a source split from every valid proper base forest of the output.

The construction uses the two-color characterization of proper forest-basedness from [1, Lemma 3.1]. Its new constraint gadget consists entirely of binary vertices. Two consecutive reticulations force a chosen color to equal a supplied variable color. Combining two such constraints below one tree vertex realizes the not-all-equal relation. These reticulation stacks leave the tree-child subclass.

Ordinary forest-based recognition has a different component requirement. Lafond and Moulton [2] prove hardness even for binary one-root networks with three leaves. Their polynomial result for two induced paths also concerns a different witness: splitting branching trees into paths can increase the component count. Neither result establishes the restriction in Theorem 1.

2. Networks and certificates
A network is a finite simple weakly connected directed acyclic graph. Here it has exactly two roots of indegree–outdegree type $(0, 2)$. Every other vertex is a tree vertex of type $(1, 2)$, a reticulation of type $(2, 1)$, or a leaf of type $(1, 0)$. At least two leaves are required.

A **proper base forest** is a spanning subgraph consisting of exactly two directed rooted trees. Its leaves are exactly the network leaves, and each omitted network arc joins different forest components. Unary vertices are allowed in this forest. The network's two roots must be the two forest roots.

Lemma 2 (two-color characterization [1]). A network of this type has a proper base forest if and only if it has a coloring $\chi : V \to \{0, 1\}$ whose root colors differ and which satisfies:

(C1) Every nonroot has exactly one parent of its own color.

(C2) Every nonleaf has at least one child of its own color.

Proof. Color a valid forest by its two components. Every vertex except the roots has one forest parent. Every network arc within a component must be selected, since omitted arcs must cross components. This proves (C1); preservation of leaves proves (C2). Conversely, select all monochromatic arcs. Each nonroot has indegree one, and acyclicity implies that its parent chain ends at a root. Distinct root colors give two components. Condition (C2) preserves the leaf set, and every omitted arc crosses components. □

The source is 3-Uniform Set Splitting, the NP-complete source used in [1, Theorem 4.1]. An instance consists of an explicitly listed universe X and a list $\mathcal{C}$ of triples of distinct elements. A split is a map $s : X \to \{0, 1\}$ making every triple nonmonochromatic. Repeated triples, isolated elements, and empty lists are allowed. Put $n = |X|$, $m = |\mathcal{C}|$, and $n' = \max(1, n)$. For an empty universe, use one dummy variable in the construction and discard it in recovery.

3. Binary color copying and variables

A **port** is a tree vertex with one private leaf child and one arc to a designated recipient. The private leaf has its parent's color by (C1), so the port satisfies (C2) regardless of the recipient's color. This prevents a use of a copied color from imposing an additional constraint on its supply.

For $k \geq 1$ recipients, a binary caterpillar supplies k ports. Its internal branching vertices have two tree children; each port has its private leaf. All vertices along the tree have the color of its initial vertex. Including that vertex and the private leaves, the construction uses $3k - 1$ vertices. Figure 1 illustrates three uses.

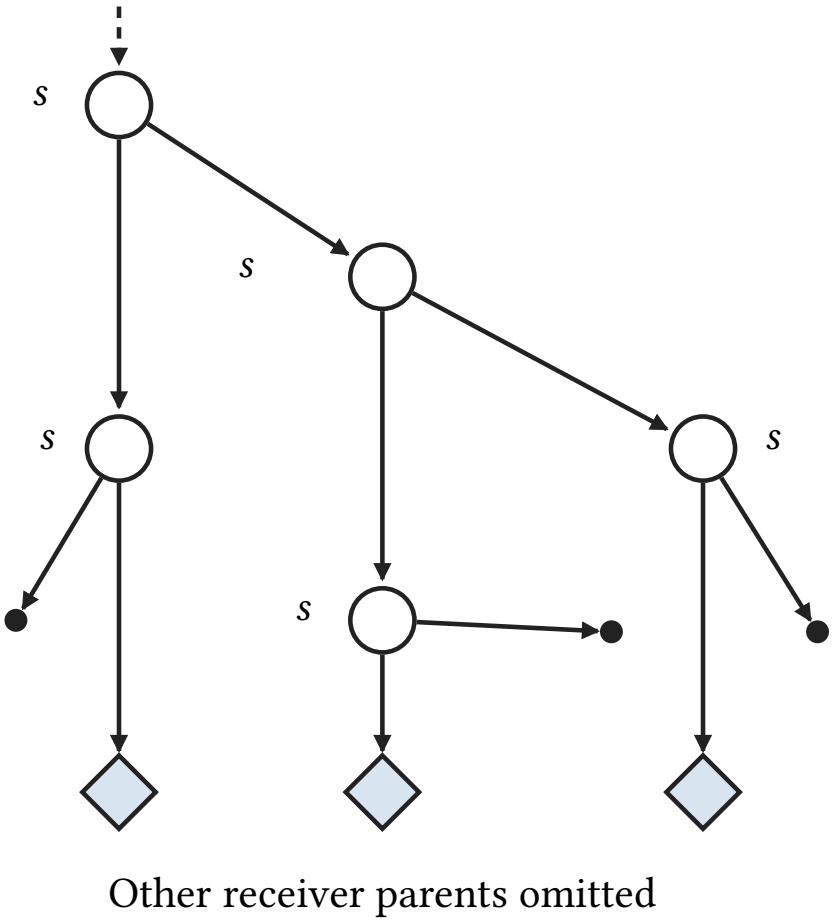


Figure 1: A three-port color-copy tree. Circles are tree vertices, diamonds are receiving reticulations, and dots are private leaves. The dashed stub indicates the incoming supply arc; the receivers' other incident arcs are omitted. Every port has a same-color private child independently of its receiver.

For each variable i, create reticulations P_i and Q_i. Each receives one port from each root's color-copy tree. Their parent colors differ, so either output color satisfies (C1). Each reticulation has a color-copy tree as its only child, which satisfies its child condition.

Create a comparison reticulation U_i with one port from each of these two color-copy trees and one leaf child. With two colors, (C1) at a reticulation is possible precisely when its two parents have different colors. Therefore P_i and Q_i are forced to have opposite colors. Either color is allowed at U_i. We interpret P_i as the source bit and Q_i as its complement. Every variable color-copy tree has at least this comparison use; each root has $2n'$ uses and therefore outdegree two.

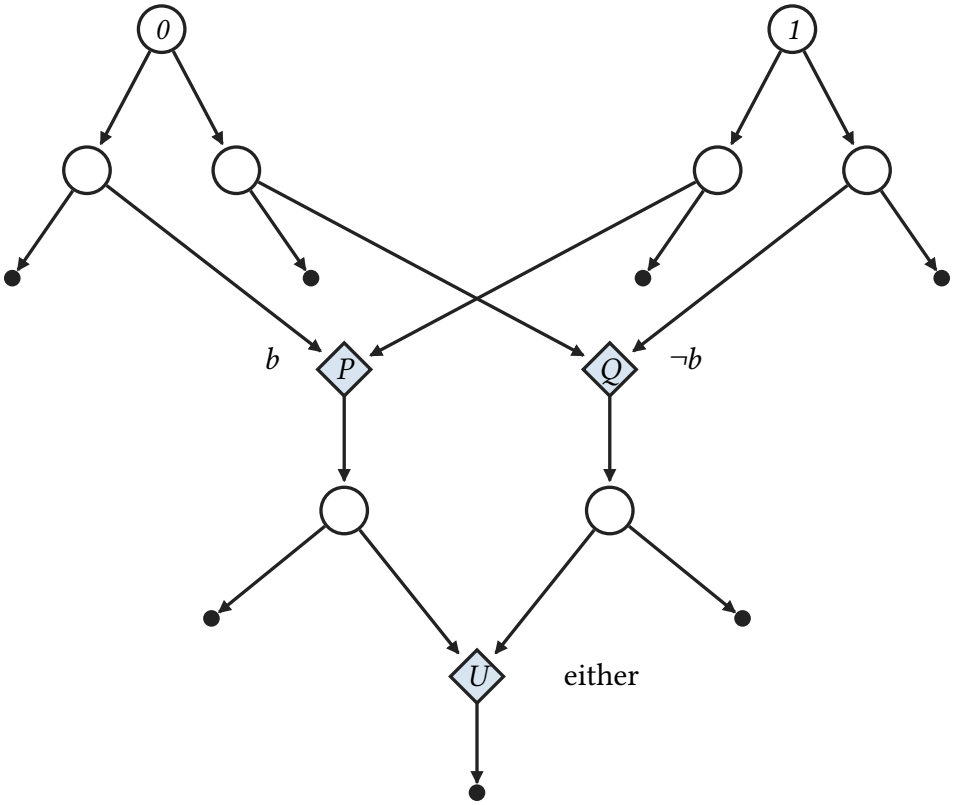


Figure 2: The complete network for one variable and no triples: 18 vertices and 19 arcs. Circles are roots or tree vertices, diamonds are reticulations, and dots are leaves. Root colors are 0 and 1; the comparison at U forces P and Q to have opposite colors. The color of U is free. This diagram has no omitted arcs.

The one-variable network in Figure 2 also handles an empty source universe. It connects the two root descendant systems through a variable reticulation, so this boundary case remains weakly connected.

4. A binary not-all-equal gadget

Consider a triple (a, b, c) and write $\alpha = s(a)$, $\beta = s(b)$, and $\gamma = s(c)$. A fresh tree vertex A receives its parent from the complement copy of a. Its children B and C are reticulations, each with a separate second parent from the positive copy of a. Reticulation B has only child D, whose second parent comes from the complement copy of b. Reticulation C has only child E, whose second parent comes from the complement copy of c. Both D and E have a leaf child. All five supply ports are distinct. Figure 3 shows this attachment.

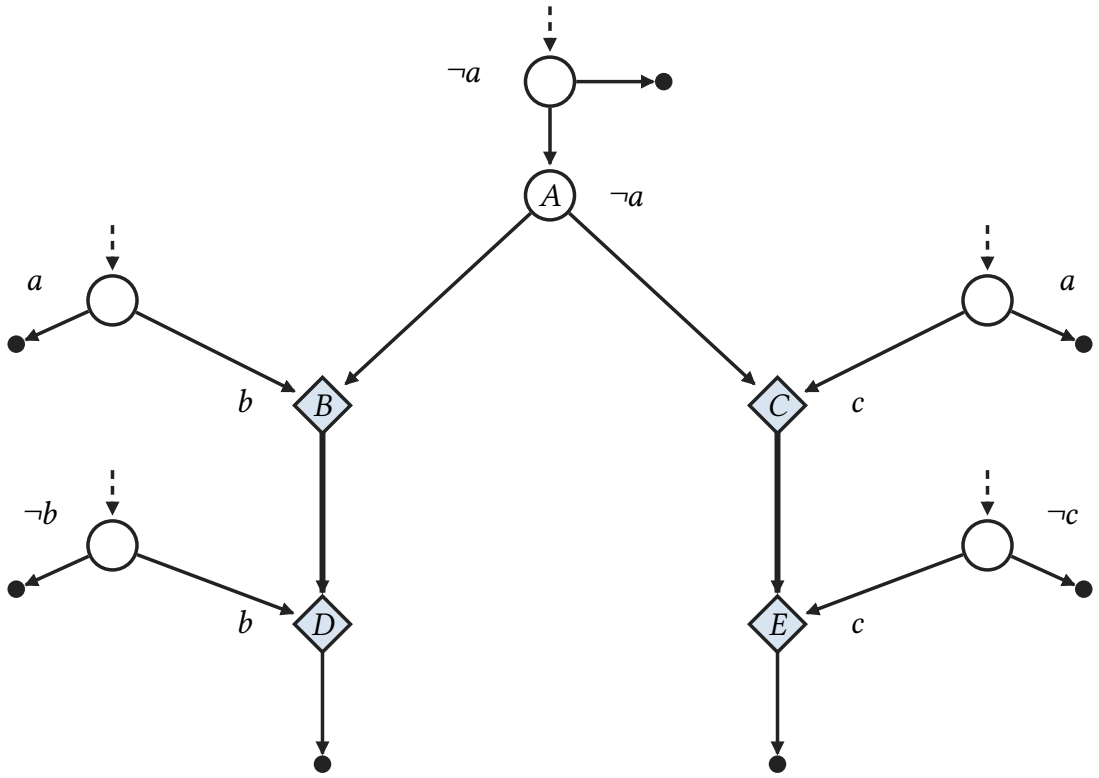


Figure 3: The clause gadget with its five supply ports and their private leaves. Circles are tree vertices, diamonds are reticulations, and dots are leaves; dashed incoming stubs connect to the relevant color-copy trees. Bold arcs mark the reticulation stacks. Labels a, b, c in the drawing denote source bits. The child condition at A is exactly the not-all-equal constraint.

Lemma 3. Given the supplied variable colors, the clause gadget extends to a coloring satisfying (C1) and (C2) if and only if the triple is nonmonochromatic.

Proof. The color of A is $1 - \alpha$. Both B and C have parents of opposite colors, so their parent conditions alone allow either color. Since D is the only child of B, condition (C2) forces their colors to agree. The two parents of D must have opposite colors, forcing the color of B to be β. The same argument applies to C and E. Thus

$$\chi(A) = 1 - \alpha, \quad \chi(B) = \chi(D) = \beta, \quad \chi(C) = \chi(E) = \gamma.$$

The remaining condition at A is

$$(1 - \alpha = \beta) \vee (1 - \alpha = \gamma),$$

which holds exactly when α, β, γ are not all equal. If it holds, these colors and the inherited leaf colors satisfy every local condition. Private leaves satisfy the child conditions of all supply ports. □

5. The reduction and its correctness

Allocate variables in their listed order and a fresh clause gadget for each listed triple. Build the two root color-copy trees and each variable's positive and negative color-copy trees, assigning one port to each requested use in allocation order. Allocate vertex labels consecutively. These rules define a deterministic forward map f.

Lemma 4. Every well-formed source instance produces a network in the required target class, regardless of its answer.

Proof. Root color-copy trees precede variable reticulations in a topological order. Variable color-copy trees follow them. Comparison reticulations and clause vertices can then be placed, with each clause ordered as A, then B, C, then D, E; leaves follow their parents. There are no arcs back to an earlier

part, so the graph is acyclic. Fresh ports and gadget vertices ensure simplicity. Each vertex has one of the specified binary types, and only the two initial vertices are roots. Private leaves ensure at least two leaves. Every variable reticulation connects both root descendant systems in the underlying undirected graph; every remaining vertex is attached to these systems. The dummy variable ensures this argument also applies when $n = 0$. □

Proof of the reduction equivalence. Given a source split, color the roots 0 and 1. Color P_i by its source bit and Q_i by its complement, propagating each color through its copy tree. Choose either color at each comparison reticulation and give its leaf the same color. By Lemma 3, each clause extends to a valid coloring. All private leaves inherit their parent colors. Lemma 2 now produces a proper base forest.

Conversely, take any valid proper base forest of the output. Label its components by their roots, assigning color 0 to root 0 and color 1 to root 1. Lemma 2 applies to this coloring. The comparison reticulations force complementary variable colors, and Lemma 3 forces every triple to be nonmonochromatic. Reading the colors of the positive variable reticulations yields a source split. Therefore

$$(X, \mathcal{C}) \in \text{3-Uniform Set Splitting} \leftrightarrow f(X, \mathcal{C}) \in \text{Proper Forest-Based Recognition}.$$

This proves both implications. □

The backward algorithm reconstructs the deterministic vertex allocation and traverses the supplied forest from its two roots. For each actual source variable, it outputs the root label reached at P_i. The proof above covers **every valid target forest**, without imposing a canonical choice at comparison reticulations. The dummy bit is discarded. Reordering the supplied forest arcs has no effect on the result.

To make the forward map total on strings, malformed source encodings map to a fixed legal NO network. This network has vertex set $\{0, \ldots, 9\}$ and arcs

$$\{(0,2), (0,3), (2,6), (2,4), (3,7), (3,4), (4,5), (1,5), (1,8), (5,9)\}.$$

Its roots are 0 and 1. Vertices 2 and 3 inherit root 0's color, but both are parents of reticulation 4, violating (C1). Direct inspection gives the required degrees, connectivity, and acyclicity.

6. Resource bounds and NP-completeness

The construction's size can be counted exactly. The two root copy trees contribute $12n' - 2$ vertices. Variable reticulations and comparison reticulation–leaf pairs contribute $4n'$. Variable copy trees have $2n' + 5m$ total uses across $2n'$ trees, contributing $4n' + 15m$ vertices. Clause interiors contribute $7m$. Consequently,

$$|V| = 20n' + 22m - 2, \quad |E| = 23n' + 26m - 4.$$

For the arc identity, each nonroot has indegree one except for the $3n' + 4m$ reticulations, which have indegree two.

With consecutive binary labels, the output uses $O((n + m + 1) \log(n + m + 2))$ bits. Construction and witness recovery each use $O(n + m + 1)$ graph operations on valid inputs. A conservative implementation bound accounting for dictionary operations and integer handling is $O((n + m + 1)^2 \log(n + m + 2))$ bit time. Parsing arbitrary malformed strings is additionally charged to their actual encoded length; the complete map remains polynomial in that length. The explicit universe prevents a short binary declaration from requesting exponentially many variables.

A target certificate lists its forest arcs. In polynomial time one checks that it spans the vertices, consists of two rooted trees with the prescribed leaves, and has every omitted network arc crossing components. Target recognition therefore belongs to NP. Source NP-hardness and the equivalence above prove Theorem 1.

7. Scope

The reduction supplies the general binary two-root hardness boundary left open in [1]. It uses reticulation stacks, so it makes no stronger tree-child or tree-sibling claim. The coloring characterization and source hardness are established ingredients; the contribution is the binary clause construction and its composition with polynomial recovery. The inspected primary literature supports this distinction. Unpublished and unindexed work remains outside the novelty assessment.

References

[1] Katharina T. Huber, Leo van Iersel, Vincent Moulton, and Guillaume E. Scholz. Is this network proper forest-based? *Information Processing Letters* 187 (2025), 106500. doi:10.1016/j.ipl.2024.106500.

[2] Manuel Lafond and Vincent Moulton. Path partitions of phylogenetic networks. *Theoretical Computer Science* 1024 (2025), 114907. doi:10.1016/j.tcs.2024.114907.

A. Verification and reproducibility

Finite tests support the implementation; they do not replace the general proof. The prepared suite checks 9 source cases and 21 recovered witnesses. A larger suite checks all 1,024 triple families on five elements and 64 seeded seven-element instances: 1,080 YES instances, 8 NO instances, and 2,160 recoveries. The seed is 20260916. A separately implemented verifier checks 37 cases and 112 recoveries, with seed 73019 and a largest output of 644 vertices. It exhausts all target forests in 15 small cases, including NO cases. These are suite counts, not deduplicated totals.

A native independent reviewer wrote its own incoming-edge enumeration and forest predicates without importing the candidate or original checkers. It exhausted the empty-universe, one-variable, two-variable, and one-triple three-variable outputs, finding respectively 4, 4, 16, and 48 forests. All 72 recoveries passed. Separate Fano-plane tests supplied one additional recovery and checked NO cases. The review found no blocking proof issue. Agent review and finite enumeration do not establish universal novelty or substitute for human expert assessment.

The implementation uses Python 3.14.7. Tests use Z3 5.1.0 and NetworkX 3.6; the paper uses Typst 0.15.1. Source JSON has fields `elements` (the explicit list of integers from zero) and `triples`. Target JSON has `vertices` and `arcs`; a target witness is a list of forest arcs. From the campaign's `work` directory, the two rule modes are:

```
python3 algorithm.py < source.json > target.json
python3 algorithm.py --extract < recovery.json > split.json
```

Here `recovery.json` contains `source` and `target_solution`. Valid target witnesses are the recovery premise. The forward mode sends malformed source values and malformed JSON to the fixed legal NO network. Fault tests reject an always-NO forward map and an all-zero extractor; malformed cyclic forests terminate with an explicit error.

From the repository root, run:

```
p=campaigns/binary-two-root-proper-forest
python3 "$p/work/check.py" --self-test
python3 "$p/work/check.py" --candidate "$p/work/algorithm.py"
python3 "$p/rounds/001/exhaustive-check.py"
python3 "$p/work/verify.py" --candidate "$p/work/algorithm.py"
python3 "$p/reviews/initial/independent_check.py"
```

Raw results are stored in `rounds/001` and `reviews/initial/check-output.json`; coverage and candidate-boundary details are in `work/verification.md`. Searches through 16 September 2026 and their limitations are recorded in `question.md` and `reviews/initial/review.md`, including checks of earlier omni-extension characterizations and an ILP formulation. Figure adjacency checks are retained in `work/figures`; the variable drawing matches the actual one-variable output. Rebuild the paper from `work` with:

```
typst compile manuscript.typ manuscript.pdf
```